透明・ポスト量子ゼロ知識証明の教科書

ZK-STARK, Ligero, Virgo, Aurora の仕組み・性能・ユースケース比較

初学者向けの直観から、暗号プロトコル設計上の比較軸まで

作成日: 2026-04-23

本書の読み方

本書は問題集ではなく、技術選定と研究導入のための解説書である。
数式は必要最小限にしつつ、性能比較では漸近量、具体ベンチマーク、実装上の制約を分けて説明する。
引用は本文中の角括弧ラベルで示し、巻末に一次論文・公式文書を掲載する。

目次

- 第 1 章 何を比較するのか - 4 方式の位置づけ
- 第 2 章 ゼロ知識証明の基礎
- 第 3 章 多項式・符号・IOP という共通言語
- 第 4 章 アリスメタイズ: AIR, R1CS, GKR, MPC-in-the-head
- 第 5 章 ZK-STARK
- 第 6 章 Ligero
- 第 7 章 Virgo
- 第 8 章 Aurora
- 第 9 章 性能比較
- 第 10 章 ユースケース比較
- 第 11 章 例で理解するモデリング
- 第 12 章 実装・パラメータ設計の実務
- 第 13 章 限界、危険な誤解、研究課題
- 用語集
- 参考文献

注: PDF の目次は静的な読み物として構成している。ページ番号よりも章タイトルを手掛かりに読む設計にした。

第 1 章 何を比較するのか

4 方式の位置づけと比較の前提

ZK-STARK, Ligero, Virgo, Aurora は、いずれも「透明」なゼロ知識証明、すなわち特定の秘密を含む信頼済み設定を必要としない方向の代表例である。楕円曲線ペアリングや回路固有の toxic waste に依存する古典的な一部の zkSNARK と対照的に、これらは主としてハッシュ関数、Merkle 木、Reed-Solomon 符号、多項式検査、ランダムオラクルモデル上の非対話化などにより構成される。

ただし、この 4 方式は同じ問題を同じ表現で解いているわけではない。ZK-STARK は実行トレースと AIR を中心に大規模な逐次計算を扱う。Ligero は NP 検証回路に対して sqrt(回路サイズ) 程度の通信を狙う MPC-in-the-head 系の簡潔な構成である。Virgo は GKR 型の層状算術回路と透明な多項式委譲により、構造化回路で検証を小さくする。Aurora は R1CS を対象に、IOP と FRI を組み合わせて透明でポスト量子的な zkSNARK を構成する。

比較で最初に確認すべきこと

「証明サイズが小さい」という言葉は、対象言語、非対話化、実装パラメータ、Merkle パス、フィールドサイズに依存する。

「検証が高速」という言葉は、検証者が回路やプログラム記述を既に持つか、公開入力が大きいか、オンチェーン検証かで意味が変わる。

「透明」と「ポスト量子」は、万能の安全性ではない。ハッシュ関数の安全性、ランダムオラクルモデル、Fiat-Shamir/BCS 変換の仮定、実装の健全性が残る。

表 1. 4 方式の粗い分類

方式	自然な計算モデル	中核アイデア	主な強み	注意点
ZK-STARK	AIR / 実行トレース / VM / RAM 型計算	トレースを低次数多項式に拡張し、FRI と Merkle コミットメントで低次数性と制約充足を検査する。	大規模逐次計算、透明性、ハッシュベース、実運用事例。	証明はペアリング系より大きくなりやすい。AIR 化とメモリ引数が性能を左右する。
Ligero	NP 検証回路、Boolean または算術回路	MPC-in-the-head/IKOS 変換を最適化し、interleaved Reed-Solomon 符号で開示量を抑える。	単純、ハッシュベース、trusted setup なし、中規模回路やバッチに強い。	通信は概ね sqrt(N) 型なので極大規模では polylog 型に漸近で劣る。
Virgo	層状算術回路、構造化 GKR 対象	GKR で層ごとに主張を縮約し、zkVPD で多項式評価を透明に委譲する。	構造化回路では検証が小さい。Merkle 木の検証実験で高速な検証。	2025 年の再検討論文は zkVPD 設計・実装上の脆弱性を報告しており、古い実装の利用には注意が必要。
Aurora	R1CS	R1CS 用 IOP、単変量 sumcheck、FRI、BCS 変換。	透明な R1CS zkSNARK、証明サイズ O(log^2 n)、R1CS との接続が明確。	検証者が制約系を読むコストは O(n)。主に研究・プロトタイプの文脈で評価される。

本書では、次の比較軸を一貫して用いる。第一に計算モデル、第二に証明サイズと通信量、第三に prover/verifier の時間・メモリ、第四に trusted setup の有無、第五に仮定の種類、最後にユースケースで

ある。特に、AIR と R1CS と層状回路は相互変換できる場合があるものの、変換が制約数、次数、メモリ、公開入力を増やすため、単純なビッグ O 比較はしばしば誤解を生む。

1.1 「公平な比較」の難しさ

ゼロ知識証明システムの比較は、暗号方式の比較であると同時にコンパイラ、フィールド演算、ハッシュ関数、実行モデル、ハードウェア、実装成熟度の比較でもある。例えば、SHA-256 の preimage 証明では Boolean 回路向けに設計された方式が強いことがある。いっぽう、長い VM 実行や rollup の状態遷移では、AIR によるトレース表現と並列 FFT/FRI を持つ STARK 系が自然である。

また、「1 制約」は方式によって同じ重さを持たない。R1CS の 1 制約、AIR の 1 行遷移、GKR の 1 ゲート、Boolean 回路の 1 ゲートは、実装上のフィールドサイズ、メモリアクセス、hash arithmetization、lookup の有無によりコストが変わる。したがって本書の性能表は、絶対的なランキングではなく、技術選定の地図として読むべきである。

1.2 本書の結論を先取りする

- 大規模な逐次計算、VM、rollup、メモリを含む計算の整合性なら、ZK-STARK が最も自然な候補である。
- 小から中規模の一般 NP 証明や、単純さ・ハッシュベース・trusted setup なしを重視するなら、Ligero は理解しやすく実装もしやすい候補である。
- 構造化された層状算術回路、特に GKR が効く問題なら、Virgo の設計思想は強力である。ただし古い Virgo/zkVPD 実装の安全性は再検討が必要である。
- R1CS を自然言語として持ち、透明かつポスト量子的な zkSNARK を研究・比較したいなら、Aurora が基準点になる。

第 2 章 ゼロ知識証明の基礎

完全性・健全性・ゼロ知識・非対話化

2.1 役者と命題

ゼロ知識証明では、prover P が「ある主張 x が言語 L に属する」こと、または「公開値 x に対して秘密 witness w が存在して関係 R(x, w)=1 を満たす」ことを verifier V に納得させる。典型例は「このハッシュ値 h に対して、h = SHA256(w) となる秘密 w を知っている」「この口座残高更新は規則に従うが、個々の取引内容は隠す」などである。

証明の本質は、秘密 witness そのものではなく、w が存在するという計算的事実を公開することである。暗号学ではこのような証明を、関係 R、公開入力 x、秘密 witness w、セキュリティパラメータ lambda、健全性誤差 epsilon により定式化する。

2.2 3 つの基本性質

表 2. ゼロ知識証明の基本性質

性質	意味	直観的な破れ方
完全性 completeness	正しい witness を持つ honest prover は、honest verifier を高確率で受理させられる。	本当に正しいのに、検証者が頻繁に reject する。
健全性 soundness	偽の主張に対して、どんな不正 prover も検証者を受理させる確率が小さい。	w がないのに、悪意ある prover が transcript を作れてしまう。
ゼロ知識 zero knowledge	証明 transcript から、主張の真偽以外の witness 情報が漏れない。	証明中の開示値や乱数の相関から、秘密入力の一部が復元される。

本書の 4 方式はいずれも「argument」と呼ばれることが多い。argument では prover は計算量的に制限され、健全性は情報理論的ではなく計算量的仮定に基づく。Merkle 木の衝突耐性やランダムオラクルモデルが典型的な仮定である。

2.3 public coin と Fiat-Shamir/BCS

多くの透明 ZK 構成は、まず対話的な IOP や public-coin プロトコルとして設計される。public coin とは verifier のメッセージが公開ランダム性であり、秘密の検証鍵を使わないことを意味する。この性質は、Fiat-Shamir 変換や IOP 用の BCS 変換により、ランダムオラクルモデルで非対話化する際に重要である。

非対話化では、prover が前のメッセージ列をハッシュし、その出力を verifier のランダムチャレンジとして使う。これにより、一つの証明文字列を誰でも検証できる。しかし、ランダムオラクルモデルや量子ランダムオラクルモデルでの安全性は微妙であり、「ハッシュすれば必ず安全」と考えてはならない。トランスクリプトのドメイン分離、順序、seed の扱い、再利用禁止は実装上の重要事項である。

直観: ゼロ知識証明は「計算の領収書」

通常の計算結果は「私は計算した」と主張するだけである。ZK 証明は、入力の秘密部分を出さずに「規則通りに計算した」という検査可能な領収書を添える。

ただし領収書の形式は方式ごとに違う。STARK は長い計算履歴の低次数性を検査する。Aurora は R1CS の充足性を検査する。Virgo は GKR で主張を層ごとに縮約する。Ligero は MPC 視点からランダムに覗いて矛盾を見つける。

2.4 succinctness と scalability

succinct という言葉は文献により少し異なる。一般には、証明サイズまたは検証時間が計算サイズ N よりずっと小さいことを指す。proof size が O(log^2 N) であれば非常に短いが、検証者が制約系そのものを O(N) 読まなければならないなら、検証時間は完全には succinct ではない。Aurora の abstract は証明サイズ O(log^2 n)、prover O(n log n)、verifier O(n) と明記している [Aurora19]。

scalability は、計算規模を大きくしたときに prover と verifier のコストがどう増えるかを表す。STARK 論文は、透明 ZK で検証がデータサイズより指数的に速くなる実装を報告し、FRI などの IOP 技術を用いる [STARK18]。実務では、proof generation のメモリ、FFT/FRI の並列化、ハッシュ回数、オンチェーン verifier のガスなどが scalability の具体的な中身になる。

2.5 ゼロ知識と witness hiding

低次数多項式や codeword の一部を開示すると、開示点の組み合わせから witness が漏れる可能性がある。ゼロ知識化では、ランダム多項式を足す、マスク行を追加する、commitment 前に乱数化する、MPC views をランダム化するなどの手法を使う。重要なのは、検査に必要な情報と witness を復元するのに必要な情報を分離することである。

方式の性能比較では、ゼロ知識化のための余分な列、余分な次数、余分な開示がしばしば見落とされる。研究プロトタイプで「non-ZK モード」が速い場合、その数字をプライバシー用途に直接持ち込むのは危険である。

第 3 章 多項式・符号・IOP という共通言語

Reed-Solomon, Merkle, FRI, sumcheck

3.1 なぜ多項式なのか

透明 ZK の多くは、「大きな計算が正しい」という主張を「ある多項式が低次数であり、いくつかの点で制約を満たす」という主張に変換する。多項式が使われる理由は、少数のランダム点での検査が強い情報を与えるからである。次数 d の 2 つの異なる多項式は、有限体上で高々 d 点でしか一致しない。したがってランダム点で一致を検査すると、不正を高確率で発見できる。

この考えは、Schwartz-Zippel 型の補題、sumcheck protocol、Reed-Solomon 符号の近接検査に支えられる。ZK-STARK と Aurora は FRI を使って Reed-Solomon codeword への近さ、すなわち「低次数多項式の評価表に近いか」を検査する。Ligero も interleaved Reed-Solomon 符号を使う。Virgo は GKR と多項式委譲を組み合わせる。

3.2 Reed-Solomon 符号と低次数拡張

有限体 F と評価領域 D を選ぶ。次数 d 未満の多項式 f を D 上で評価したベクトル (f(x))_{x in D} が Reed-Solomon codeword である。計算トレースや witness ベクトルを低次数多項式で補間し、より大きい領域 D' 上で評価する操作を low degree extension (LDE) と呼ぶ。STARK 論文は LDE と composition degree を、prover 時間・空間の主要ボトルネックとして扱っている [STARK18]。

LDE により、局所的な制約を多項式恒等式として表しやすくなる。例えば Fibonacci 列なら、trace の隣接行に対して a_{i+1} - b_i = 0、b_{i+1} - a_i - b_i = 0 という遷移制約を課す。これを多項式上の条件に変換し、境界条件と組み合わせて一つまたは少数の composition polynomial にまとめる。

3.3 Merkle commitment

IOP の prover は巨大な oracle 文字列を送る代わりに、その oracle を Merkle 木に commit する。検証者はランダムに選んだ少数の位置を問い合わせ、prover はその位置の値と authentication path を返す。これにより、通信は「開示値 + Merkle パス」に圧縮される。

この構造は透明で実装しやすいが、証明サイズでは Merkle パスのハッシュ値が大きな成分になる。STARK 論文は、行を同じ Merkle subtree に置くなど、authentication path complexity を下げる工夫を説明している [STARK18]。

3.4 FRI: 低次数性を折り畳んで検査する

FRI は Fast Reed-Solomon Interactive Oracle Proof of Proximity の略である。FRI は、ある長いベクトルが Reed-Solomon codeword に近いことを、folding を繰り返しながら小さな問題へ還元する。直観的には、評価表を半分の長さに畳み、そのたびにランダム線形結合を使って不正な高次数成分を隠れにくくする。

FRI 論文は、prover の算術計算量が厳密に線形、verifier が厳密に対数、かつ定数健全性を持つ RS-IOPP として FRI を提示した [FRI18]。現代の STARK 実装では DEEP-FRI などの改良がよく使われ、評価領域の外側の点をサンプリングすることで健全性解析を強める [DEEPFRI20]。

FRI の非常に粗い流れ

```
oracle w_0 は「低次数多項式の評価表」だと主張される。
1. verifier がランダム alpha_0 を選ぶ。
2. prover は w_0 を alpha_0 で fold し、短い oracle w_1 に commit する。
3. これを log 回ほど繰り返し、最後は低次数性を直接検査できる大きさにする。
```

```
4. verifier は各層の対応位置を少数開き、fold の整合性と最後の多項式を検査する。
```

3.5 sumcheck と GKR

sumcheck protocol は、多変量多項式 g の hypercube 上の総和がある値 H に等しいことを対話的に検査する古典的手法である。各ラウンドで一変数多項式を送り、検証者はランダム点に主張を縮約する。GKR プロトコルは、層状算術回路の出力値の主張を、sumcheck を使って一つ下の層の値に還元し、これを入力層まで繰り返す。

Virgo はこの GKR の構造を使う。層数 D、ゲート数 C、入力数 n の回路で、構造化されていれば verification time を O(D log C + log^2 n) にできる、というのが Virgo の中核的な主張である [Virgo20]。ただし検証者が回路構造を簡潔に計算できることが前提であり、任意の巨大回路をただ渡すだけではそのままの検証量にならない。

第 4 章 アリスメタイズ

AIR, R1CS, 層状回路, MPC-in-the-head

4.1 アリスメタイズとは

アリスメタイズとは、プログラムや回路の正しさを有限体上の代数的制約へ変換する工程である。ZK 証明の性能は、しばしば暗号プロトコル本体よりもアリスメタイズにより決まる。ハッシュ関数を何制約で表せるか、メモリ読み書きをどう検査するか、range check をどう行うか、lookup を使うか、フィールドと計算の相性はどうか、といった問題が全体のコストを左右する。

4.2 AIR: 実行トレース中心の表現

AIR (Algebraic Intermediate Representation) では、計算を時系列の表、すなわち trace として表す。各行が時刻、各列がレジスタやメモリ補助値を表す。制約は「次の行との関係」「境界条件」「公開入力との一致」などとして書かれる。STARK 系では AIR が自然である。

Fibonacci の AIR 例

```
trace columns: a, b
boundary: a_0 = 0, b_0 = 1
transition for i = 0..T-2:
  a_{i+1} - b_i = 0
  b_{i+1} - a_i - b_i = 0
public output:
  b_{T-1} = y
```

AIR の強みは、同じ遷移制約が長い trace に繰り返し適用される点である。CPU/VM の 1 ステップ制約、STARK-friendly ハッシュの round、状態遷移などは AIR と相性がよい。一方で、不規則なメモリアクセス、ランダムアクセス配列、複雑な条件分岐を効率よく表すには permutation/memory argument が必要になる。

4.3 R1CS: Rank-1 Constraint Satisfaction

R1CS は witness ベクトル z に対して、各制約が (A_i z) * (B_i z) = (C_i z) という形を持つ表現である。多くの zkSNARK ツールチェーンはプログラムを R1CS に変換する。Aurora はこの R1CS を直接対象とし、R1CS の satisfiability を透明な IOP により証明する [Aurora19]。

R1CS は汎用的であるが、1 制約が 1 つの乗法を表すため、Boolean 演算やビット分解のコストが大きくなりがちである。SHA-256 のようなビット演算中心の計算は制約数が膨らむ。逆に、有限体上の算術を自然に使う計算や回路コンパイラが整った環境では扱いやすい。

4.4 層状算術回路と GKR

層状算術回路は、入力層から出力層までゲートが層ごとに並び、各層のゲートが前の層の値に基づいて計算する回路である。GKR では、この構造を使って出力の正しさを 1 層ずつ縮約する。Virgo はこの対象に特化することで、構造化回路で小さい検証を得る。

GKR が効くには、各層の配線やゲート種別を verifier が簡潔に評価できることが重要である。例えば FFT のような規則的配線、行列演算、反復構造を持つ計算は候補になる。一方で、任意の回路を巨大な表として持ち込むと、回路記述を読むコストがボトルネックになり得る。

4.5 MPC-in-the-head と Ligero

MPC-in-the-head では、prover が秘密 witness を使って仮想的な MPC プロトコルを頭の中で実行し、複数 party の view を commit する。verifier はランダムに一部の view を開かせ、整合性を検査する。全ての view を開けば witness が漏れるが、一部だけならゼロ知識を保てる。

Ligero はこの一般変換を最適化し、MPC 通信と Reed-Solomon 符号を組み合わせて通信量を sqrt(回路サイズ) 程度に抑える。論文は任意の collision-resistant hash を使え、ランダムオラクルモデルで非対話化でき、trusted setup や public-key cryptography を不要とする構成を示す [Ligero17]。

表 3. 計算表現と 4 方式

表現	向く計算	代表方式	コストに効く要素
AIR	長い逐次計算、VM、状態遷移、反復ハッシュ	ZK-STARK	trace 幅、trace 長、遷移次数、LDE blowup、メモリ引数。
R1CS	一般 NP、既存 zkSNARK コンパイラの対象、算術回路	Aurora	制約数、行列の疎性、公開入力、ビット演算の arithmetization。
層状算術回路	規則的な配線を持つ並列計算、GKR が効く計算	Virgo	深さ D、ゲート数 C、入力数 n、回路構造の簡潔性。
検証回路 + MPC view	中規模 NP 証明、ハッシュベースで単純な証明	Ligero	回路サイズ、party 数、RS 符号パラメータ、開示 view 数。

第 5 章 ZK-STARK

Scalable Transparent ARgument of Knowledge

5.1 概要

ZK-STARK は、透明性、スケーラビリティ、ハッシュベースの仮定を同時に狙う証明システムである。原論文は、透明な ZK システムで検証がデータサイズより指数的に速くなる実装を報告し、DNA profile match というユースケースで、データベースや候補者の DNA 情報を隠したまま「一致しない」ことを示す proof-of-concept を示した [STARK18]。

名前の要素は、Scalable、Transparent、ARgument of Knowledge である。Scalable は大規模計算に対して prover がほぼ線形または準線形、verifier と proof size が小さいことを目指す。Transparent は trusted setup が不要で、公開ランダム性からパラメータを決められることを意味する。Knowledge は、受理される証明から witness または計算履歴を抽出できる意味での知識性を意図する。

5.2 プロトコルの流れ

ZK-STARK の典型的な pipeline

```
program / VM / computation
  -> execution trace
  -> AIR constraints: boundary, transition, memory, public input
  -> low degree extension of trace columns
  -> composition polynomial: constraints are folded into one low-degree claim
  -> Merkle commitments to oracle evaluations
  -> FRI / DEEP-FRI low-degree test
  -> Fiat-Shamir or public randomness -> non-interactive proof
```

最初のステップは、計算を trace 表にすることである。各レジスタ列を低次数多項式で補間し、より大きい領域に評価する。次に、全ての制約違反がゼロであることを、一つの composition polynomial の低次数性またはゼロ性へまとめる。verifier は全体を読まず、ランダムに少数の行や FRI の対応点を開かせて検査する。

STARK の証明サイズには、開示されたフィールド要素、Merkle authentication path、FRI 層の commitment、チャレンジ、場合によっては DEEP 点や補助値が含まれる。実装では、hash function の出力長、Merkle tree の arity、field の選択、クエリ数、blowup factor が証明サイズを決める。

5.3 AIR と composition polynomial

AIR 制約は局所的である。例えば「次行は前行から規則に従って計算される」「最初の行は公開入力に一致する」「最後の行は公開出力に一致する」といった条件である。これを多項式にするには、trace 列を多項式 f_j にし、遷移制約 C(f_j(x), f_j(gx), ...) が評価領域上でゼロになることを要求する。

複数の制約を別々に検査すると通信が増える。そこでランダム係数で線形結合し、composition polynomial にまとめる。もしどれかの制約が大きく破れていれば、ランダム線形結合が偶然ゼロになる確率は小さい。これが、代数的ランダム化により多数の条件を一つの低次数性主張へ圧縮する基本技法である。

5.4 FRI と transparent setup

STARK は polynomial commitment として楕円曲線ベースの KZG ではなく、Merkle commitment と FRI を使う。prover は codeword を Merkle 木に commit し、verifier はランダムに少数点を開く。FRI が低次数性を検査し、Merkle 木が oracle の一貫性を保証する。

このため、STARK の trusted setup は不要である。ただし、非対話化された STARK はランダムオラクルモデルに基づく。interactive STARK なら verifier の公開ランダム性を直接使えるが、ブロックチェーンや一般公開証明では単一の証明文字列が望まれるため、Fiat-Shamir/BCS 風の非対話化が使われる。

5.5 性能の特徴

STARK の prover は trace 生成、LDE、composition polynomial 計算、FRI の各層計算、Merkle hashing に多くの時間を使う。prover コストは trace サイズにほぼ比例または準線形に増える。verifier は全体の trace を読まず、ランダムクエリと Merkle path と FRI 検査を行うため、計算全体に比べて非常に小さい。

原論文は、加法 FFT による厳密な準線形 prover と、検証が厳密に対数的になる STIK/STARK 構成を説明している [STARK18]。また、SHA2, AES, DPM などで composition degree を下げる工夫を示し、authentication path の最適化も議論している。

STARK の強み

大規模な逐次計算の証明に向く。計算サイズが大きいほど、全体を再計算しない検証の価値が出る。

trusted setup が不要で、ハッシュ関数を中心とした仮定にできる。

実運用では StarkEx, Starknet, SHARP などの文脈で、STARK 証明と再帰・集約が使われている。Starknet の SHARP 文書は Stone/S-two prover と再帰モデルを説明している [StarknetSHARP]。

STARK の弱み・設計上の注意

証明サイズは Groth16 のようなペアリング系と比べて大きくなりやすい。Ethereum.org も STARK の透明性・ポスト量子性と、証明サイズが大きいという欠点を説明している [EthereumZKRollups]。

AIR 化が難しい計算、特に不規則メモリや複雑なデータ構造では、メモリ制約がボトルネックになる。

hash choice は重要である。SHA-256 はビット演算としては重い場合があり、STARK-friendly hash が選ばれることが多い。

5.6 ユースケース

- ZK rollup / validity rollup: 多数の取引をオフチェーンで実行し、状態遷移の正しさを STARK proof でオンチェーン検証する。
- VM 実行証明: Cairo や RISC-V 風 VM の実行 trace を証明し、任意プログラムを扱う。
- 大規模データベース検査: DNA profile match のように、データベースとクエリの秘密性を保ちながら、探索や非一致を証明する。
- 規制・監査: 取引や計算ログを公開せずに、規則準拠だけを第三者に検証させる。
- 機械学習推論の検証: 大きな推論計算を trace 化できる場合、出力の正しさを検証する。ただし量子化、lookup、行列演算の効率化が必要。

STARK は「全てに最小の証明を与える方式」ではなく、「大規模な計算の完全性を、透明で比較的保守的な暗号仮定に基づき証明する方式」と理解するとよい。

第 6 章 Ligero

Lightweight Sublinear Arguments Without a Trusted Setup

6.1 概要

Ligero は、NP のゼロ知識 argument で、通信量が検証回路サイズの平方根に比例することを狙う方式である。原論文は、任意の collision-resistant hash function に基づけること、ランダムオラクルモデルで非対話化できること、trusted setup と public-key cryptography を不要とすることを特徴として挙げる [Ligero17]。

Ligero は STARK や Aurora のような FRI 中心の polylog 証明サイズを目指すものではない。代わりに、MPC-in-the-head と Reed-Solomon 符号を組み合わせ、設計を単純にし、現実的な中規模回路で短い通信を実現する。

6.2 MPC-in-the-head の見方

秘密 witness を持つ prover は、仮想的に複数 party の MPC を実行し、各 party の view を作る。これらの view は、計算が正しければ互いに整合する。prover は全 view に commit し、verifier はランダムに選んだ view を開かせる。多くの view を開けば witness が漏れるため、ゼロ知識を保つように開示パターンを設計する。

Ligero の鍵は、party 数を roughly sqrt(circuit size) に設定すると、1 party あたりの通信も roughly sqrt(circuit size) になるという発想である。論文は underlying MPC の総通信が回路サイズ程度で、party 数に依存しない点を利用する [Ligero17]。

6.3 interleaved Reed-Solomon 符号

Ligero は witness や MPC view を行列として扱い、列や行に Reed-Solomon 符号的な構造を与える。verifier はランダムな行・列・線形結合を開かせ、ローカルな整合性と回路制約を検査する。interleaving により、個別の符号語を大量に送るのではなく、複数の符号語をまとめて検査する。

この設計は、PCP/IOP の抽象的な低次数検査を、実装しやすい行列・符号検査として捉え直したものといえる。Ligero 論文は「simple zero-knowledge interactive PCP based on interleaved Reed-Solomon codes」と説明している [Ligero17]。

Ligero の粗い protocol flow

```
verification circuit C, public input x, witness w
  -> prover simulates a tailored MPC protocol in its head
  -> prover arranges views / wire values in an encoded matrix
  -> prover commits to rows/columns via hashes
  -> verifier sends public random challenges
  -> prover opens selected rows/columns and consistency checks
  -> Fiat-Shamir makes the protocol non-interactive in ROM
```

6.4 concrete performance

原論文の abstract は、SHA-256 preimage のゼロ知識検証について、soundness error 2^-40 の場合に通信量が約 44KB、また plausible conjecture の下で 34KB 未満、prover 140ms、verifier 62ms と報告している [Ligero17]。後の拡張版は、よりタイトな解析、改善実装、形式的証明を含み、SHA-256 preimage で約 35KB と述べる [Ligero23]。

これらの数字は、当時の実装・パラメータ・対象回路に対するものであり、現代のハードウェアやハッシュ関数、コンパイラと単純に比較してはいけない。しかし、Ligero の設計目標が「巨大計算のための最先端 rollup prover」ではなく、「trusted setup なしで軽量な sublinear argument を作る」ことだった点はよく分かる。

6.5 amortization

Ligero は同じ検証回路に対する多数インスタンスを同時に証明する場合に、amortized communication を大きく下げられる。原論文は、multi-instance setting でサブ線形検証時間と改善された通信量を得ると述べ、SHA-256 の多数インスタンスでインスタンスあたり通信が非常に小さくなる例を示す [Ligero17]。

これは、同じ circuit C を何度も使う用途で重要である。例えば、多数のパスワードハッシュ、複数の署名検証、同じ検査規則での多数レコード監査などで、単発証明よりもバッチ証明が有利になる。

Ligero の強み

構成が比較的単純で、collision-resistant hash のみで説明しやすい。

trusted setup が不要で、非対話化もランダムオラクルモデルで可能。

中規模回路や同一回路の多数インスタンスでは、実用的な通信量を得やすい。

Ligero の弱み・注意点

単発証明の通信量は sqrt(N) 型で、N が非常に大きくなると STARK/Aurora の polylog 型に漸近で劣る。

verifier の計算量や hashing は対象実装に依存し、オンチェーン検証の主流候補としては STARK/SNARK 系に比べて採用例が少ない。

soundness 2^-40 の数字は現代の高セキュリティ用途には小さい場合があり、2^-80 や 128-bit では通信が増える。

6.6 ユースケース

- SHA-256 preimage やパスワード関連証明: 既存ハッシュを隠したまま知識を証明する。
- 同一規則の大量レコード監査: 各レコードが規則を満たすことをバッチで証明する。
- 教育・研究プロトタイプ: trusted setup なしの sublinear argument を比較的少ない道具で実装・理解する。
- ポスト量子を意識した軽量 proof-of-knowledge: 楕円曲線ペアリングを避けたいが、極大規模 STARK までは不要な場合。

Ligero を選ぶときは、「証明をどれだけ短くしたいか」だけでなく、「どの程度単純な仮定と実装で十分か」「同じ circuit を多数回使うか」を重視する。

第 7 章 Virgo

Transparent Polynomial Delegation and GKR

7.1 概要

Virgo は、層状算術回路に対する透明な succinct zero-knowledge argument である。原論文は、D-depth、n inputs、C gates の回路に対して、prover time O(C + n log n)、proof size O(D log C + log^2 n)、構造化回路では verification time も O(D log C + log^2 n) と述べる [Virgo20]。

Virgo の背後には、透明な zero knowledge verifiable polynomial delegation (zkVPD) と GKR がある。GKR は回路の出力主張を入力層まで縮約するが、その過程で多項式評価の正しさが必要になる。Virgo はこれを trusted setup なしで実現するために zkVPD を導入する。

7.2 GKR による層ごとの縮約

層状回路の出力値が正しいことを示すには、各層の gate が前層の値から正しく計算されていることを検査すればよい。しかし全ゲートを読むと検証が大きくなる。GKR は、層 i の値に関する主張を sumcheck により層 i-1 の少数の値に関する主張へ縮約する。これを繰り返すと、最後には入力層の値に関する主張になる。

この縮約は、回路配線が多項式として簡潔に表せる場合に特に強い。Virgo が「structured circuit」で verifier time も succinct と述べるのはこのためである [Virgo20]。

7.3 zkVPD: 多項式評価を透明に委譲する

VPD は、prover が大きなベクトルまたは多項式を commit し、その評価値 f(r) が正しいことを短い証明で verifier に納得させる仕組みである。pairing-based VPD なら短くできるが、trusted setup や楕円曲線仮定が入る。Virgo は透明な zkVPD を導入し、collision-resistant hash など軽量プリミティブで構成する。

Virgo 論文は zkVPD について、O(N log N) prover、O(log^2 N) proof/verification を述べる。また、実装は Mersenne prime fields を使い、Merkle tree 256 leaves を計算する回路で proof generation 53 秒、verification 50ms、proof size 253KB と報告している [Virgo20]。

Virgo の粗い protocol flow

```
layered arithmetic circuit C
  -> GKR reduces output claim layer by layer using sumcheck
  -> claims become evaluations of multilinear extensions
  -> zkVPD commits to/evaluates the needed polynomials
  -> Merkle/hash commitments provide transparency
  -> final proof certifies the circuit computation without trusted setup
```

7.4 性能の特徴

Virgo は、prover time の実験で既存の succinct ZK argument より高速と報告された一方、proof size は数百 KB になり得る。これは STARK や Aurora と同様に、透明性を得るために Merkle path やハッシュベースの証明を使うためである。Virgo 論文自身も、proof size は pairing-based VPD より大きいと述べる [Virgo20]。

Virgo の性能は回路構造に強く依存する。GKR が効く回路なら verifier は小さい。逆に、構造を簡潔に記述できない回路、深さが大きすぎる回路、メモリ消費が過大な実装では利点が弱まる。論文は現在の実装が single-threaded であり、並列化可能だがメモリ消費が大きいことにも触れている [Virgo20]。

7.5 重要な安全性注記: Revisiting Virgo

2025 年に公開された「Revisiting Virgo」は、Virgo の zkVPD protocol design and implementation に、見落とされていた制限と複数の exploitable security vulnerabilities を報告し、改善版の VPD/zkVPD を提案している [VirgoRevisit25]。このため、Virgo を歴史的・学術的方式として学ぶことと、古い Virgo 実装をそのまま高価値用途に用いることは分けて考える必要がある。

Virgo を読む際の研究者向け注意

Virgo の設計思想は GKR + transparent polynomial delegation の重要な基準点である。

しかし 2025 年以降に古い実装や古い zkVPD 設計を使う場合、Revisiting Virgo の指摘と修正版を確認する必要がある。

「透明」「ポスト量子らしい」「論文で実験済み」は、実装安全性の十分条件ではない。

7.6 ユースケース

- 構造化回路の委譲計算: 規則的な層状回路で大きな計算を証明する。
- permissioned blockchain での privacy-preserving computation: 原論文は proof size が permissioned blockchain では許容され得ると述べる。
- Merkle tree やデータ構造演算の証明: 実験例として Merkle tree 256 leaves が示される。
- GKR 系 proof system の研究基準: Libra, deVirgo, distributed proof systems などを読む前の基礎になる。

Virgo は、実用選定では最新の修正・実装状況を慎重に確認すべき方式である。一方、GKR と透明多項式委譲の結合を理解するうえでは非常に有用な教材である。

第 8 章 Aurora

Transparent Succinct Arguments for R1CS

8.1 概要

Aurora は、R1CS の satisfiability を対象とする透明な zkSNARK である。ePrint abstract は、transparent setup、plausibly post-quantum secure、lightweight cryptography を特徴とし、n constraints の証明サイズ O(log^2 n)、prover O(n log n)、verifier O(n) と述べる [Aurora19]。128-bit security で数百万 constraints でも証明が 130KB 未満と報告されている。

Aurora の重要性は、R1CS という多くの zkSNARK ツールが使う標準的表現に対して、trusted setup なしで比較的小さい証明を与える点にある。STARK が AIR/実行トレースに自然なのに対し、Aurora は R1CS を直接対象にする。

8.2 R1CS を IOP へ変換する

R1CS では、各制約が (A z) * (B z) = C z という形を持つ。Aurora は、witness と各線形形式の評価を Reed-Solomon codeword として扱い、R1CS の全制約が満たされることを IOP で検査する。大きな witness 全体を送るのではなく、oracle として commit し、ランダムな位置や線形結合だけを開く。

Aurora の中核には、古典的 sumcheck の単変量 analogue がある。ePrint abstract は、Reed-Solomon codeword の subgroup 上の和を検証する IOP を key ingredient として挙げる [Aurora19]。この IOP と FRI により、低次数性と R1CS 充足性が検査される。

Aurora の粗い protocol flow

```
R1CS instance: A, B, C, public x
witness z
  -> encode z and linear evaluations as Reed-Solomon codewords
  -> IOP checks R1CS consistency via univariate sumcheck
  -> FRI checks low-degree proximity
  -> BCS / random oracle compiles IOP to a non-interactive SNARG
  -> proof contains commitments, openings, FRI transcripts
```

8.3 Aurora と FRI

Aurora は、low-degree testing のために FRI を使う。FRI を使うことで trusted setup は不要になり、proof はハッシュ/Merkle ベースの transcript になる。これは STARK と共通するが、対象言語と arithmetization が異なる。STARK は AIR と execution trace を低次数化する。Aurora は R1CS の witness/制約評価を codeword として扱う。

Aurora 論文は、同様の透明・ポスト量子性を持つ prior zkSNARK と比べて proof size を大きく短縮したと述べる。また、Ligero との比較では、Ligero の O(sqrt N) 型 query/communication に対し、Aurora は query complexity を O(log N) 型へ下げる一方、round complexity や IOP 構成が複雑になる。

8.4 検証時間 O(n) の意味

Aurora の verifier は O(n) とされる。これは、R1CS の制約系が任意に与えられる場合、検証者が制約系の記述にアクセスし、それに応じたチェックを行う必要があるためである。つまり、proof size は短くても、verifier time は完全に polylog ではない。

この点はユースケースに直結する。もし verifier が固定された回路を既に知っており、制約系の digest や preprocessed description を利用できるなら、実務上の検証体験は改善する。しかし、任意の巨大 R1CS をその場で公開して検証するなら、制約系を読むコストが残る。

Aurora の強み

R1CS という広く使われる表現を対象にする。

trusted setup なしで、plausibly post-quantum な zkSNARK を構成する。

証明サイズは O(log^2 n) で、論文は 128-bit security で数百万 constraints でも 130KB 未満と報告する。

Aurora の弱み・注意点

verifier は O(n) であり、任意 R1CS では制約系を読むコストが残る。

実用 rollup などでは、近年の STARK/Plonkish/FRI 系プロトコルや PCS との比較が必要である。

R1CS 化で制約数が膨らむ計算、特にビット演算中心の計算では、別の表現が有利な場合がある。

8.5 ユースケース

- R1CS ベースの研究比較: transparent setup の R1CS zkSNARK として基準になる。
- 既存 R1CS toolchain との接続: witness generation が R1CS 前提なら、Aurora の考え方が自然である。
- ポスト量子性を重視する汎用 NP 証明: pairing-based SNARK を避けたい研究・長期機密用途で検討対象になる。
- 教育用途: IOP、FRI、BCS 変換、R1CS の関係を一つの方式で学べる。

第 9 章 性能比較

漸近量、具体数字、設計判断

9.1 比較表

表 4. 性能比較の要約

方式	証明/通信の典型像	prover	verifier	代表的な公開数字
ZK-STARK	非対話 proof は開示点と Merkle path と FRI transcript。方式・パラメータにより数十 KB からそれ以上。理論的には検証・proof size が polylog 型と説明されることが多い。	trace 生成 + LDE + FRI + hashing。計算サイズに対して準線形が目標。	全 trace を読まず、ランダム開示と FRI/Merkle を検査。大規模計算で強い。	原論文は透明 ZK で指数的検証高速化を報告。DNA profile match proof-of-concept を提示 [STARK18]。
Ligero	概ね O~(sqrt(N)) communication。soundness を上げると増える。	MPC view 生成、RS 符号化、hash commitment。	単発では回路サイズ依存の処理が残りやすい。バッチでは amortized 改善。	SHA-256 preimage, 2^-40 soundness で約 44KB、prover 140ms、verifier 62ms [Ligero17]。
Virgo	O(D log C + log^2 n) proof size for D-depth structured layered circuits。実験では数百 KB。	O(C + n log n)。GKR と zkVPD のコスト。	構造化回路で O(D log C + log^2 n)。	Merkle tree 256 leaves 回路で proof generation 53 秒、verification 50ms、proof size 253KB [Virgo20]。
Aurora	O(log^2 n) proof size for R1CS。論文では 128-bit で数百万 constraints でも 130KB 未満。	O(n log n) field operations。	O(n)。制約系を読むコストが残る。	R1CS zkSNARK として transparent setup, plausibly post-quantum [Aurora19]。

表 4 の数字は同じベンチマークではない。特に Ligero の 2^-40 soundness と Aurora の 128-bit security は直接比較できない。Virgo の 253KB は特定の Merkle tree 回路に対する実験値であり、STARK の rollup proof と同じ対象ではない。

9.2 証明サイズ

証明サイズだけで見ると、ペアリング系 SNARK が数百バイト級になる一方、透明・ハッシュベース方式は数十 KB から数百 KB 以上になりやすい。これは Merkle path と FRI/IOP transcript を含むためである。しかし、trusted setup を避けたい場合、量子耐性を重視する場合、大規模計算で prover/verifier のスケーリングが重要な場合、証明サイズの大きさは受け入れられることがある。

Aurora は R1CS に対して O(log^2 n) proof size を持つ。Virgo も構造化回路で O(D log C + log^2 n) を主張する。STARK はパラメータにより具体サイズが変わるが、polylog 型の検証と proof size が技術的目標である。Ligero は sqrt(N) 型で、N が中程度なら現実的だが、N が非常に大きいと polylog 型に劣る。

9.3 prover 時間

prover 時間は、証明システムの実用性を最も左右する。ブロックチェーン rollup では、prover が大量の取引を一定時間内に処理できる必要がある。企業監査や privacy-preserving query では、prover が秘密データ所有者側にあり、メモリや電力の制約を受けることがある。

STARK prover は LDE/FRI/hashing が重いが並列化しやすい。Ligero prover は対象回路と MPC view 生成に依存し、中規模では軽量な利点がある。Virgo prover は GKR + zkVPD で O(C + n log n) とされるが、メ

モリや実装最適化が重要である。Aurora prover は O(n log n) field operations で、R1CS サイズの増加がそのまま効く。

9.4 verifier 時間

verifier 時間はユースケースにより重みが異なる。オンチェーン検証では verifier の gas が支配的であり、証明サイズとハッシュ回数もコストになる。サーバー間の監査では、数十 ms から数秒の検証は許容されるかもしれない。ブラウザやモバイル検証では、証明サイズと CPU がともに問題になる。

STARK と Virgo は、構造化された設定で検証を非常に小さくできる。Aurora は proof size は短いが verifier が O(n) であり、任意 R1CS の全制約読み込みが問題になる。Ligero はバッチ化で amortized verifier を改善できるが、単発では回路サイズ依存が残りやすい。

9.5 メモリ

メモリは論文の抽象的なビッグ O に現れにくいが、実装では致命的である。STARK は trace と LDE codeword を保持するため、巨大なメモリを消費しやすい。現代実装では streaming、chunking、parallel commitment、recursion/aggregation、GPU/FPGA 最適化が重要になる。Virgo 論文も大きなメモリ消費を課題として挙げている [Virgo20]。

Ligero と Aurora でも、巨大な witness、行列、codeword を持つとメモリが問題になる。多項式証明は「短い証明」を作るが、その前段階で prover は大きなデータ構造を扱う。prover がクラウドで動くのか、ユーザーデバイスで動くのかは方式選定に直結する。

9.6 実装成熟度

表 5. 実装成熟度の観点

方式	成熟度の見方	実装上の観点
ZK-STARK	StarkWare/Starknet などで実運用に近いエコシステムがある。SHARP は STARK proofs の生成・集約・再帰を扱う公式文書がある [StarknetSHARP]。	Cairo/VM、Stone/S-two、recursive proving、オンチェーン verifier、DA との統合。
Ligero	研究実装・プロトタイプとして理解しやすい。商用大規模 rollup での主流というより、軽量 transparent argument の基準。	MPC view、hash commitment、バッチ証明、soundness repetition。
Virgo	GKR 系 transparent zkSNARK の重要な研究基準。ただし 2025 年の脆弱性再検討に注意 [VirgoRevisit25]。	修正版 zkVPD、構造化回路、メモリ、並列化、安全な実装確認。
Aurora	IOP/FRI ベース R1CS zkSNARK の重要な研究基準。libiop という IOP-oriented library が論文で紹介される [Aurora19]。	R1CS toolchain、FRI parameters、BCS transcript、verifier O(n)。

第 10 章 ユースケース比較

どの方式をどの場面で検討するか

10.1 ユースケース行列

表 6. ユースケース比較

ユースケース	最有力候補	理由	代替・注意
Validity rollup / L2	ZK-STARK	長い状態遷移を trace として証明しやすく、STARK 系の実運用エコシステムがある。	オンチェーン検証コストと proof size を再帰・集約で下げる。
R1CS ベースの汎用証明研究	Aurora	R1CS を直接対象にし、transparent setup と FRI/IOP の関係が明確。	verifier O(n) と R1CS 化コストに注意。
中規模 hash preimage 証明	Ligero	SHA-256 preimage で具体的な小さい通信と高速実験が報告されている。	高い security level では repetition と通信増を考慮。
構造化層状回路	Virgo	GKR により層ごとに主張を縮約できる。	Revisiting Virgo の指摘を踏まえ、修正版と実装検証が必要。
巨大データベースの秘密監査	ZK-STARK / Ligero	STARK は大規模逐次処理、Ligero は規則的な多数インスタンスに向く。	データアクセス pattern をどう隠すか、漏洩モデルを別途検討。
教育・プロトタイピング	Ligero / Aurora	MPC-in-the-head と IOP/FRI をそれぞれ学ぶ教材としてよい。	本番用途の安全性は実装とパラメータを精査。
長期ポスト量子志向	4 方式全て候補	いずれも楕円曲線ペアリングへの依存を避ける方向。	「plausibly post-quantum」であり、QROM とハッシュ security parameter を確認。

10.2 ブロックチェーン

ブロックチェーンでは、証明サイズ、verifier gas、データ可用性、再帰・集約、prover latency が一体となって重要になる。Ethereum.org は ZK-STARK について、transparent で CRS trusted setup なし、量子計算機に対しても安全と考えられる一方、証明サイズが大きく Ethereum 上の検証費用が高くなり得ると説明している [EthereumZKRollups]。

STARK はこの領域で実運用文脈を持つ。Starknet/SHARP の文書は、Stone/S-two prover、再帰 proof、onchain verification への送信などのアーキテクチャを説明する [StarknetSHARP]。Ligero, Virgo, Aurora は、概念・研究比較として重要だが、2026 年時点の主流 L2 実装を選ぶ観点では STARK 系や Plonkish/FRI 系の現代実装と比較する必要がある。

10.3 プライバシー保護監査

企業や行政が秘密データを保持しつつ、規則準拠だけを証明したい場合、ゼロ知識証明は有力である。STARK 論文の DNA profile match はその典型例で、データベースも候補者の DNA profile も開示せず、非一致を証明する [STARK18]。

この種の用途では、計算自体の秘匿よりも、アクセスパターン、公開入力、証明頻度、証明を誰が生成するかが重要になる。例えば「全データを scan した」ことを証明するなら逐次計算として STARK が合う。多数レコードが同じ規則を満たすことをバッチで示すなら Ligero の amortization も検討できる。

10.4 機械学習・データ処理

機械学習推論の証明では、行列乗算、非線形関数、量子化、range check、lookup、メモリアクセスが問題になる。STARK は大規模 trace と並列化に強いが、モデルが巨大だと proof generation と memory が重い。Virgo/GKR 系は規則的な回路構造に強い可能性がある。Aurora は R1CS 化できるが、制約数が膨らみやすい。

実務では、どの proof system を選ぶか以前に、モデルをどの数値表現に落とすか、誤差許容をどう扱うか、秘密は重みか入力か出力か、証明頻度はどの程度かを決める必要がある。

10.5 個人認証・資格証明

「資格を満たすが、生年月日や ID は出さない」「パスワードを知っているがパスワードは出さない」という用途では、proof size と verification UX が重要である。Ligero のような hash preimage に強い方式は理解しやすいが、モバイル端末で prover を走らせるには最適化が必要である。

より広い anonymous credentials では、signature proof や set membership、revocation をどう扱うかが中心になり、4 方式単体ではなく、コミットメント、署名、accumulator、Merkle tree、range proof との組み合わせが必要になる。

第 11 章 例で理解するモデリング

Fibonacci, SHA-256, Merkle tree, rollup

11.1 例 1: Fibonacci の出力を証明する

公開値 T と y に対し、「Fibonacci 列の T 番目が y である」ことを証明したい。これは単純だが、各方式の計算表現の違いを理解するのに向く。

- ZK-STARK: trace に a_i, b_i を置き、隣接行遷移 a_{i+1}=b_i, b_{i+1}=a_i+b_i を AIR 制約にする。長い T に対して自然である。
- Aurora: 各ステップの変数を R1CS witness に置き、線形制約・乗法制約で遷移を表す。加算だけなら R1CS としては軽いが、変数数は T に比例する。
- Virgo: 層状回路として Fibonacci を表すこともできるが、逐次依存が強いので深さが T になりやすい。GKR の利点は小さい。
- Ligero: 検証回路 C が「T ステップの Fibonacci を計算して y と比較する」とし、その circuit satisfiability を証明する。

11.2 例 2: SHA-256 preimage

公開値 h に対して、秘密 w が SHA256(w)=h を満たすことを証明する。これはゼロ知識証明の典型例であるが、ビット演算中心であるため算術化の選択が重要である。

- Ligero: 原論文が具体ベンチマークとして SHA-256 preimage を扱う。Boolean 回路ベースで説明しやすい。
- ZK-STARK: SHA-256 を AIR で表すことは可能だが、STARK-friendly hash に比べるとコストが重い場合がある。
- Aurora: R1CS に SHA-256 を落とすとビット制約が多くなる。既存 R1CS ツールがあれば自然だが制約数に注意。
- Virgo: SHA-256 を層状回路にできるが、構造化回路として GKR の利点を十分引き出せるかは設計次第。

11.3 例 3: Merkle tree membership

秘密の leaf と authentication path が、公開 root に対応することを証明する。ブロックチェーン、認証、データベース監査で頻出する。

- ZK-STARK: hash round を trace として扱い、path の各段を AIR で証明する。大量の membership をまとめると有利になる。
- Ligero: hash circuit の検証として扱う。中規模単発や複数インスタンスのバッチで検討できる。
- Virgo: 原論文の実験例に Merkle tree 256 leaves の回路があり、53 秒の proof generation、50ms verification、253KB proof size が報告される [Virgo20]。
- Aurora: hash を R1CS 化し、R1CS satisfiability として証明する。proof size は短いが制約系の読み込みがある。

11.4 例 4: ZK rollup の状態遷移

rollup では、前状態 root、取引バッチ、後状態 root が公開される。秘密または圧縮された取引データに対して、全取引が署名・残高・nonce・手数料・Merkle 更新規則を満たすことを証明する。

この用途では、計算は長い逐次処理であり、メモリやデータ可用性も関わる。ZK-STARK は自然な候補である。Starknet/SHARP のように、複数プログラムや複数ブロックの proof を集約し、再帰でオンチェーン検証コストを下げる設計が使われる [StarknetSHARP]。

Aurora や Ligero で rollup を表すことも理論上は可能だが、巨大な R1CS/回路の扱い、オンチェーン verifier、proof aggregation で別の課題が出る。Virgo は構造化回路に強いが、不規則な状態遷移とメモリ更新を効率化できるかが鍵である。

11.5 例 5: 規制準拠の秘密監査

金融機関が「全ての取引が制限ルールを満たす」「ある顧客に不正な重複登録がない」ことを証明したいが、顧客情報や取引内容を開示したくないとする。この場合、秘密データを witness とし、規則を circuit/AIR として表す。

データ件数が非常に多く、同じ規則を全件 scan するなら STARK が向く。各レコードの独立検査を多数インスタンスとしてまとめるなら Ligero も考えられる。R1CS toolchain で規則を簡単に記述できるなら Aurora の設計が参考になる。構造化集計回路なら Virgo/GKR 系の発想が効く場合がある。

第 12 章 実装・パラメータ設計の実務

安全性、性能、運用のチェックリスト

12.1 security parameter と soundness error

soundness error は、偽の証明が受理される確率の上界である。2^-40 は研究プロトタイプや比較実験では見かけるが、高価値の金融用途では不足することが多い。128-bit security を目標にするなら、クエリ数、field size、hash output、FRI repetitions、Fiat-Shamir transcript の扱いを一貫して設計する必要がある。

複数の証明を合成したり、同じ system を大量に運用したりする場合、union bound により全体の失敗確率を見積もる。単発で 2^-80 でも、10^12 回の証明を検証するシステムでは安全余裕が小さくなる。

12.2 ハッシュ関数の選択

透明 ZK ではハッシュ関数が Merkle commitment と Fiat-Shamir transcript の中核になる。オンチェーンでは EVM precompile やガスの制約、STARK prover では field-friendly hash の制約、汎用セキュリティでは SHA-2/SHA-3 などの保守性が効く。

hash を「暗号学的に安全」とだけ見てはいけない。ZK 回路内でハッシュを再帰検証する場合、hash の arithmetization cost が証明全体を支配する。STARK-friendly hash は field arithmetic と相性がよいが、標準化や暗号解析の成熟度との trade-off がある。

12.3 Fiat-Shamir transcript

非対話証明では、transcript が仕様そのものである。commitment の順序、public input の encoding、domain separation tag、protocol version、field element の canonical encoding、Merkle root の encoding を明確にする。これらが曖昧だと、異なる実装間で検証結果が不一致になったり、malleability や challenge reuse の問題が出る。

また、Fiat-Shamir の random oracle 入力には、全ての前段 commitment と公開文脈を含めるべきである。proof aggregation や recursion では、内側 proof の transcript と外側 proof の transcript が混ざらないように domain separation が必須である。

12.4 フィールド選択

有限体 F の選択は、FFT/FRI の効率、セキュリティ、回路表現、実装容易性に影響する。大きな 2-adicity を持つ prime field は FFT に向く。binary field はビット演算や一部 STARK で自然な場合がある。Mersenne prime は高速な modular reduction に利点がある。

一方で、field size が小さすぎると soundness 解析が弱くなる。拡大体を使えば解決できることがあるが、prover/verifier の field operation が重くなる。field と hash と arithmetization を別々に選ぶのではなく、全体最適化が必要である。

12.5 メモリと並列化

prover の最大メモリは、proof generation の可否を決める。trace 全体を保持する設計は簡単だが、大規模計算では破綻する。streaming trace、chunked commitment、out-of-core FFT、GPU memory tiling、distributed proving などが必要になる。

STARK は計算構造上、FFT/FRI/hashing を並列化しやすい。Virgo/GKR 系も層ごとの gate 評価や sumcheck の一部を並列化できる。Ligero の MPC view と hash commitment も並列化可能である。Aurora の R1CS 行列演算も疎行列処理として最適化できる。

12.6 実装検証

- 仕様と実装の差分をテストする。特に transcript encoding と public input binding は重要である。
- soundness parameter を固定し、テスト用の低 security parameter と本番用を混同しない。
- fuzzing と property-based testing で、malformed proof、異常な field element、Merkle path 長不一致を検査する。
- constant-time が必要な箇所を特定する。proof verification は公開入力中心だが、prover は witness を扱う。
- 依存する hash、field arithmetic、FFT、serialization のバージョンを固定し、reproducible build を用意する。

12.7 ベンチマークの作法

ベンチマークでは、同じ安全性レベル、同じ計算、同じ公開入力、同じハードウェア、同じスレッド数、同じ hash 関数を明示する。prover time には witness generation を含めるか、proof system 部分だけかを分ける。memory peak、proof size、verification time、setup/preprocessing time、recursion cost を別々に報告する。

特に論文間比較では、年代が違うと CPU、コンパイラ、SIMD、GPU、hash 実装が大きく変わる。古い論文の数値を現代実装の絶対ランキングに使うのではなく、設計上のスケーリングと当時の技術的達成として読むべきである。

第 13 章 限界、危険な誤解、研究課題

透明 ZK は万能ではない

13.1 危険な誤解

- 誤解 1: trusted setup がないなら安全である。実際には hash、random oracle、実装、パラメータ、arithmetization の安全性が残る。
- 誤解 2: post-quantum と書かれていれば量子攻撃に完全に安全である。実際には「楕円曲線離散対数に依存しない」程度の意味で使われることも多く、QROM と security parameter の確認が必要である。
- 誤解 3: proof size が短い方式が常に良い。prover time、verifier time、memory、回路表現、deployment cost が同時に重要である。
- 誤解 4: ゼロ知識なら何も漏れない。公開入力、証明タイミング、サイズ、失敗有無、アクセスパターン、side channel は別問題である。

13.2 研究課題

第一の課題は recursion と aggregation である。大量の証明をまとめ、オンチェーンまたは軽量 verifier に渡すには、内側 proof を外側 proof で検証する必要がある。STARK 系では recursive STARK や pairing-based compression などの設計があるが、hash と field の arithmetization が課題になる。

第二の課題は lookup と memory argument である。実世界のプログラムは配列、テーブル、範囲制約、条件分岐を多用する。これを AIR/R1CS/GKR に効率よく落とすには、lookup protocol、permutation argument、RAM consistency proof が不可欠である。

第三の課題は実装検証である。proof system は数学的に正しくても、serialization、transcript、field reduction、randomness、parallelization のバグで破れる。Virgo の再検討論文は、著名な透明 ZK 方式でも設計・実装の再検査が重要であることを示している [VirgoRevisit25]。

13.3 4 方式から学ぶ設計原理

表 7. 4 方式から抽出できる設計原理

設計原理	ZK-STARK	Ligero	Virgo	Aurora
計算表現を合わせる	AIR/VM に合う計算で強い。	検証回路として表しやすい。	層状・構造化回路で強い。	R1CS に合う計算で強い。
透明性のコストを理解する	Merkle/FRI で proof が大きめ。	hash commitment と repetition。	transparent zkVPD で proof が大きめ。	FRI/BCS transcript。
低次数性を使う	trace と制約を低次数多項式にする。	RS 符号で view/witness を検査。	多項式評価委譲と GKR。	R1CS IOP と単変量 sumcheck。
実装依存性	prover 最適化とメモリが重要。	単純だが security level で通信増。	構造と安全な zkVPD が鍵。	R1CS サイズと verifier O(n) が鍵。

13.4 最後の判断指針

新しい用途で proof system を選ぶときは、次の順序で考えるとよい。まず、秘密にしたいものと公開してよいものを定義する。次に、計算をどのモデルに自然に落とせるかを確認する。第三に、必要な security level と運用回数から soundness を設計する。第四に、prover が許容できる時間・メモリを見積もる。第五に、verification の場所、オンチェーンかサーバーかクライアントかを決める。最後に、実装成熟度と監査状況を評価する。

ZK-STARK, Ligero, Virgo, Aurora は、どれも透明 ZK の発展に重要な役割を持つ。しかし、それぞれが別の最適化点を持つ。方式名ではなく、計算モデルと運用制約から選ぶことが、暗号学者としての堅実な比較である。

付録 A 共通コンポーネント詳解

Merkle, FRI, sumcheck, transcript をもう一段深く読む

A.1 Merkle path のサイズ感

Merkle commitment は透明 ZK の実装を単純にするが、proof size では無視できない。葉が M 個で二分木なら、1 点を開くために約 log2(M) 個のハッシュ値が必要になる。各ハッシュが 32 bytes なら、1 query あたり 32 log2(M) bytes が authentication path だけで必要になる。多点開示では path の共有により圧縮できるが、ランダム点が広く散ると共有は限定的である。

STARK 論文が row placement や affine coset query を意識するのは、field element の数だけでなく Merkle path の重複を減らすためである。実装者は、query complexity と hash output length だけで proof size を見積もらず、Merkle tree arity、batch opening、path compression、root の個数を含めて計算する必要がある。

Merkle opening の概算

```
binary tree with M leaves
one opening:
  value bytes + log2(M) * hash_bytes
q independent openings:
  q * value bytes + roughly q * log2(M) * hash_bytes
batched openings:
  fewer than q * log2(M) hashes if paths share nodes
practical proof size = algebraic data + FRI data + Merkle paths + transcript metadata
```

A.2 FRI folding の直観を数式で見る

FRI の 1 ラウンドを非常に単純化して書く。評価領域 D が対になっており、x と -x が同じ coset に属するとする。prover は codeword f(x) を偶部分 f_even(x^2) と奇部分 x f_odd(x^2) に分解し、verifier のランダム alpha に対して f_next(y)=f_even(y)+alpha f_odd(y) を commit する。

もし f が低次数なら f_next も低次数である。もし f が低次数から遠いなら、ランダム alpha によりその遠さが次ラウンドにも高確率で残る。これを繰り返すと、長い codeword の低次数性が短い codeword の低次数性へ還元される。実際の FRI は領域、folding、query consistency、soundness 解析がより精密である。

A.3 sumcheck の小さな例

2 変数多項式 g(X1, X2) について、H = sum_{b1,b2 in {0,1}} g(b1,b2) を証明したい。prover はまず h1(X1)=sum_{b2 in {0,1}} g(X1,b2) を送る。verifier は h1(0)+h1(1)=H を確認し、ランダム r1 を選ぶ。次に prover は h2(X2)=g(r1,X2) を送る。verifier は h2(0)+h2(1)=h1(r1) を確認し、ランダム r2 を選ぶ。最後に g(r1,r2) を直接または別の委譲手段で確認する。

GKR はこの sumcheck を各層で使う。層 i の全ゲートに関する大きな総和主張を、ランダム点での前層の値に関する主張へ縮約する。Virgo はこの最後の「ランダム点での値」を zkVPD で透明に証明する。

A.4 transcript binding のミスはなぜ危険か

Fiat-Shamir/BCS 非対話化では、challenge は transcript の hash である。もし public input の一部、回路 digest、Merkle root、protocol version を hash 入力から落とすと、同じ proof を別文脈へ流用できる可能性が生じる。これは proof の malleability や replay につながる。

したがって、実装仕様には「challenge_i = H(domain, protocol_version, statement_digest, previous_messages, commitment_i)」のように、どの byte 列をどの順序で hash するかを厳密に書く。field element は canonical representation を使い、複数表現を許さない。

A.5 ゼロ知識シミュレータの直観

ゼロ知識性は、実際の witness を知らない simulator が、検証者から見て本物と区別できない transcript を作れることで示される。STARK/Aurora では、開示される oracle 値が witness を特定しないようにランダム多項式や masking を使う。Ligero では、開示される MPC view だけから witness が復元できないように秘密分散的な構造を使う。Virgo では zkVPD が評価値を証明しつつ多項式の不要な情報を隠す。

シミュレータの存在は抽象的に見えるが、実装では「開示点が多すぎないか」「同じ randomness を再利用していないか」「複数 proof を組み合わせると secret が解けないか」という検査項目になる。

付録 B 方式別プロトコル・スケッチ

論文を読むための擬似コード

B.1 ZK-STARK prover

STARK prover sketch

```
input: program P, public input x, witness w
1. Execute P(x,w) and build trace table T.
2. Check/construct AIR constraints: boundary, transition, memory.
3. Interpolate trace columns and evaluate LDE over large domain.
4. Sample Fiat-Shamir challenges for random linear combinations.
5. Build composition polynomial C that encodes all constraint violations.
6. Commit to trace LDE and C via Merkle roots.
7. Run FRI/DEEP-FRI to prove C and related oracles are low-degree.
8. Open queried positions and Merkle authentication paths.
output: roots, openings, FRI transcript, metadata
```

検証者側は、公開入力と transcript から同じ challenge を再計算し、開示された少数の trace 行で AIR 制約が composition polynomial と整合すること、FRI transcript が低次数性を示すこと、Merkle path が root に一致することを確認する。

B.2 Ligero prover

Ligero prover sketch

```
input: verification circuit C, public input x, witness w
1. Secret-share witness-dependent values into virtual MPC parties.
2. Simulate the MPC protocol for C inside the prover.
3. Arrange party views and wire values as encoded matrices.
4. Apply interleaved Reed-Solomon encoding where required.
5. Commit to rows/columns/views using collision-resistant hashes.
6. Derive random challenges from public coins or Fiat-Shamir.
7. Open selected views and consistency checks.
output: commitments, opened views, RS consistency data
```

検証者は、開示された view が互いに整合し、公開入力と出力に矛盾せず、符号検査に合格することを確認する。ランダムに選ばれなかった view は witness を隠す役割を持つ。

B.3 Virgo prover

Virgo prover sketch

```
input: layered arithmetic circuit C, input x, witness/private input w
1. Evaluate all layers of C.
2. Commit to polynomial representations of layer values using zkVPD.
3. Start with claim about output layer value.
4. For each layer from output to input:
     run GKR/sumcheck to reduce claim to previous layer evaluations.
     prove needed polynomial evaluations via zkVPD.
5. Finish at input layer and public/witness consistency.
```

```
output: GKR transcripts + zkVPD proofs + commitments
```

検証者は、各 sumcheck の整合性、回路配線多項式、zkVPD の評価証明、最終的な入力層の主張を確認する。回路が structured であれば、配線多項式を短く評価できる。

B.4 Aurora prover

Aurora prover sketch

```
input: R1CS matrices A,B,C, public input x, witness z
1. Encode z and auxiliary vectors as Reed-Solomon codewords.
2. Commit to codewords as IOP oracles via Merkle roots.
3. Run R1CS consistency checks using univariate sumcheck IOP.
4. Use random linear combinations to fold many constraints.
5. Invoke FRI to prove relevant oracles are close to low-degree codewords.
6. Compile the IOP transcript using BCS / Fiat-Shamir.
output: commitments, openings, sumcheck transcript, FRI proof
```

検証者は、R1CS instance と public input を binding した transcript から challenges を再計算する。低次数性、R1CS linear checks、opening consistency を検査する。任意 R1CS では、制約系を読むための O(n) コストが残る。

B.5 4 方式を同じ擬似コードで見る

表 9. 4 方式を共通ステップへ写像する

抽象ステップ	ZK-STARK	Ligero	Virgo	Aurora
計算を表す	trace/AIR	verification circuit + MPC views	layered circuit	R1CS
長いデータへ commit	trace LDE と composition oracle	view/wire matrices	layer polynomials / zkVPD	RS codewords
ランダム検査	AIR local check + FRI queries	randomly opened views/rows/columns	sumcheck per layer	univariate sumcheck + FRI
低次数・符号	FRI/DEEP-FRI	interleaved RS checks	zkVPD/IOP	FRI
非対話化	Fiat-Shamir/NIROP	Fiat-Shamir	Fiat-Shamir/IOP transcript	BCS transformation

付録 C ベンチマーク解釈と換算の注意

論文数値を誤読しないために

C.1 soundness level を合わせる

2^-40, 2^-80, 128-bit security は同じではない。例えば Ligero の 44KB は 2^-40 soundness の数字である [Ligero17]。Aurora の 130KB 未満は 128-bit security での報告である [Aurora19]。これらを「44KB と 130KB だから Ligero が常に小さい」と比較するのは誤りである。

soundness を上げる一般的な方法は、クエリ数を増やす、繰り返す、field size を大きくする、FRI/DEEP-FRI のパラメータを変える、hash output を長くするなどである。どの方法が最も効率的かは方式により異なる。

C.2 計算サイズ N の定義を合わせる

STARK の N は trace 長と幅、AIR 制約、LDE 領域の積に近い。Aurora の n は R1CS constraints である。Virgo の C は層状回路の gate 数、D は深さである。Ligero の N は検証回路サイズである。これらは同じ「計算」を表しても数値が大きく異なる。

例えば SHA-256 は Boolean 回路では自然だが、field arithmetic では bit decomposition が必要になる。逆に field 上の多項式計算は R1CS や AIR で軽く、Boolean 回路では重くなる。この差を無視すると、方式の優劣ではなく arithmetization の優劣を比較してしまう。

C.3 prover time に witness generation を含めるか

多くの論文は proof generation time として、証明システム部分だけを測る場合がある。実務では、witness generation、プログラム実行、trace generation、I/O、serialization、データ取得も時間に含まれる。rollup なら transaction execution と state access、監査ならデータベース scan、ML ならモデル推論が含まれる。

ベンチマークを設計する際は、end-to-end time と cryptographic proving time を分けて報告する。後者だけが速くても、witness generation が支配的ならユーザ体験は改善しない。

C.4 verifier time と verification environment

オンチェーン verifier は通常の CPU verifier とは違う。EVM で Keccak が安いか、SHA-256 precompile が使えるか、field arithmetic が高いか、calldata cost がどうかで結果が変わる。STARK proof は proof size が大きくなりがちだが、再帰や集約で onchain verifier に渡す proof を小さくする設計がある。

サーバー verifier では、数百 KB の proof と数十 ms から数秒の検証が許容される用途もある。モバイル verifier では、帯域、バッテリー、メモリ、JavaScript/WASM 実装が問題になる。

C.5 比較論文を読むテンプレート

- 対象言語: AIR, R1CS, layered circuit, Boolean circuit のどれか。
- security parameter: soundness, zero-knowledge, hash output, field size。
- proof size: Merkle path, transcript, public input, commitment を含むか。
- prover time: witness generation と proof generation を分けているか。
- verifier time: 回路記述を読む時間を含むか。オンチェーンか CPU か。
- setup: trusted setup, universal setup, transparent setup の違い。
- implementation: thread 数、CPU/GPU、compiler、field arithmetic、hash 実装。

付録 D ユースケース設計ノート

要件から方式へ落とす

D.1 Rollup 設計ノート

rollup の証明では、状態遷移関数 STF をどう表すかが第一の課題である。VM を導入すれば、任意プログラムを同じ AIR で証明できるが、VM overhead が増える。アプリ固有回路にすれば高速化できるが、柔軟性と開発コストが下がる。STARK は VM と相性がよく、Cairo のような実行環境を中心に証明する設計が成立する。

proof aggregation は必須に近い。多数ブロックや多数アプリケーションの proof を個別にオンチェーン検証するとコストが大きい。SHARP のような shared prover は、複数の Cairo programs の証明を集約し、再帰により効率化する方向を示す [StarknetSHARP]。

D.2 秘密監査設計ノート

秘密監査では、証明そのものよりデータモデルが難しい。例えば「全ての顧客が条件を満たす」と証明しても、どの顧客集合を対象にしたかをどう binding するか、途中でデータを除外していないことをどう示すかが問題になる。Merkle root、timestamp、監査対象 dataset の commitment、監査規則の versioning が必要である。

証明方式としては、全件 scan の正しさを trace 化するなら STARK、同一検査を多数インスタンスで行うなら Ligero、構造化集計なら Virgo/GKR 系、R1CS ツールで規則を書くなら Aurora 型の設計が候補になる。

D.3 ML 推論設計ノート

ML 推論では、浮動小数点をそのまま証明するのは難しいことが多い。量子化、固定小数点、lookup table、近似多項式が必要になる。ZK-friendly なモデル変換を行うと、元モデルの精度や説明責任に影響する。

STARK は大量の反復演算を trace として扱えるが、モデルサイズが巨大だと memory と proof time が問題になる。GKR 系は規則的な行列演算に向く可能性があるが、非線形活性化や lookup が課題になる。R1CS/Aurora はコンパイラが書きやすい一方、制約数が膨らみやすい。

D.4 資格証明・匿名認証設計ノート

匿名認証では、単に witness を隠すだけではなく、linkability、revocation、issuer trust、credential update が問題になる。ZK proof system はその一部であり、signature scheme、commitment、accumulator、nullifier と組み合わせて設計する。

この領域では、proof size と verifier time がユーザ体験を左右する。モバイルで prover を走らせるなら Ligero の軽量性は魅力的だが、credential の構造が R1CS に自然なら Aurora 型、Merkle inclusion が中心なら STARK 型の設計が参考になる。

D.5 データベース検索設計ノート

データベース検索では、「検索結果が正しい」だけでなく、「該当なしなら本当に全件調べた」「該当件数を隠すか公開するか」「アクセスパターンを隠すか」が問題になる。STARK 論文の DNA profile match は、公開の信頼と個人情報保護の間にある緊張を説明するよい例である [STARK18]。

検索対象が秘密で、アクセス pattern も秘密にしたいなら ORAM や private information retrieval との組み合わせが必要になる場合がある。ZK proof は計算の正しさを示すが、全ての side-channel を自動で消すわけではない。

付録 E 原論文の読み方

初学者が迷いやすい箇所

E.1 ZK-STARK 論文の読み方

まず introduction と Figure/benchmark を読み、次に AIR、ALI、FRI、LDE の関係を追う。全ての定義を最初から完全に理解しようとすると難しい。先に「trace -> constraints -> composition polynomial -> FRI」という pipeline を頭に置くとよい。

STARK 論文は、ZK-STIK と STARK の形式定義、interactive/non-interactive 変換、FRI と Merkle commitment の実装上の最適化を含む。実装者は、composition degree と authentication path complexity の議論を特に読むべきである。

E.2 Ligero 論文の読み方

Ligero は抽象的 PCP より、MPC-in-the-head の直観から入ると理解しやすい。party views を commit し、ランダムに開くことで、正しい MPC 実行を検査する。なぜ sqrt(N) になるかは、party 数と per-party communication のバランスとして読む。

interleaved Reed-Solomon code の部分は、単一の符号語ではなく、多数の関連する列をまとめて検査する手段として理解するとよい。具体ベンチマークは soundness parameter と対象回路を必ず確認する。

E.3 Virgo 論文の読み方

Virgo は、GKR を知らないと難しい。先に GKR の layer-by-layer reduction と sumcheck を学び、その後に zkVPD がどの主張を担っているかを見る。C, D, n の意味、structured circuit の仮定、proof size と verifier time の式を分けて読む。

2025 年以降は、Revisiting Virgo も併読する。特に zkVPD の設計・実装上の脆弱性指摘は、古い Virgo を安全な実装として扱う際の前提を変える。

E.4 Aurora 論文の読み方

Aurora は R1CS、IOP、FRI、BCS 変換が重なる。まず abstract の O(log^2 n), O(n log n), O(n) を覚え、次に R1CS の充足性をどの oracle と check で表すかを見る。単変量 sumcheck が key ingredient である。

Aurora と STARK の違いを理解するには、「同じ FRI を使っても、証明している低次数性の対象が違う」と考えるとよい。STARK は実行 trace の制約、Aurora は R1CS witness と線形評価の整合性を扱う。

E.5 自分で実装する最小ステップ

1. 有限体演算と多項式補間・評価を実装する。
2. Merkle commitment と batched opening を実装する。
3. 小さな Reed-Solomon codeword の低次数性検査を実装する。
4. Fibonacci AIR を作り、非 ZK の toy STARK を作る。
5. Fiat-Shamir transcript を明示的に仕様化する。
6. ゼロ知識 masking を追加し、開示点から witness が漏れないかテストする。
7. 同じ toy relation を R1CS にし、Aurora/Ligero の見方と比較する。

本番実装はここから大きく離れるが、toy 実装は論文の抽象概念を身体化するのに有効である。

用語集

表 8. 用語集

用語	説明
AIR	Algebraic Intermediate Representation。実行 trace と代数的制約で計算を表す。STARK 系で中心的。
BCS 変換	Interactive Oracle Proof をランダムオラクルモデルで非対話 argument に変換する手法。
FRI	Fast Reed-Solomon IOP of Proximity。低次数性・RS codeword 近接性を folding により検査する。
GKR	Goldwasser-Kalai-Rothblum protocol。層状算術回路の主張を sumcheck で層ごとに縮約する。
IOP	Interactive Oracle Proof。prover が oracle として大きな文字列を提示し、verifier が一部を問い合わせるモデル。
LDE	Low Degree Extension。ベクトルを多項式で補間し、より大きな領域で評価する。
Merkle commitment	データ列を Merkle 木の root に commit し、個別位置を authentication path で開示する。
R1CS	Rank-1 Constraint Satisfaction。制約 (A z)*(B z)=C z の集合で計算を表す。
soundness error	偽の主張が受理される確率の上界。
trusted setup	証明・検証パラメータ生成に秘密 randomness や trapdoor が関わる設定。透明方式はこれを避ける。
zkVPD	Zero-knowledge Verifiable Polynomial Delegation。多項式評価の正しさを秘密を隠して短く証明する仕組み。
Fiat-Shamir	対話プロトコルの verifier randomness を transcript hash で置き換え、非対話化する方法。

参考文献

[GMR89] Shafi Goldwasser, Silvio Micali, Charles Rackoff. The Knowledge Complexity of Interactive Proof Systems. SIAM Journal on Computing, 1989.

[LFKN92] Carsten Lund, Lance Fortnow, Howard Karloff, Noam Nisan. Algebraic Methods for Interactive Proof Systems. Journal of the ACM, 1992.

[GKR08] Shafi Goldwasser, Yael Tauman Kalai, Guy N. Rothblum. Delegating Computation: Interactive Proofs for Muggles. STOC 2008. URL: https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/2008-DelegatingComputation.pdf

[BCS16] Eli Ben-Sasson, Alessandro Chiesa, Nicholas Spooner. Interactive Oracle Proofs. TCC 2016 / IACR ePrint related works.

[FRI18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, Michael Riabzev. Fast Reed-Solomon Interactive Oracle Proofs of Proximity. ICALP 2018. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ICALP.2018.14

[DEEPFRI20] Eli Ben-Sasson, Lior Goldberg, Swastik Kopparty, Shubhangi Saraf. DEEP-FRI: Sampling Outside the Box Improves Soundness. ITCS 2020 / ePrint 2019/336. URL: https://eprint.iacr.org/2019/336

[STARK18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. IACR ePrint 2018/046. URL: https://eprint.iacr.org/2018/046

[Ligero17] Scott Ames, Carmit Hazay, Yuval Ishai, Muthuramakrishnan Venkitasubramaniam. Ligero: Lightweight Sublinear Arguments Without a Trusted Setup. ACM CCS 2017. DOI: 10.1145/3133956.3134104. URL: https://acmccs.github.io/papers/p2087-amesA.pdf

[Ligero23] Scott Ames, Carmit Hazay, Yuval Ishai, Muthuramakrishnan Venkitasubramaniam. Ligero: lightweight sublinear arguments without a trusted setup. Designs, Codes and Cryptography, 2023. URL: https://link.springer.com/article/10.1007/s10623-023-01222-8

[Virgo20] Jiaheng Zhang, Tiancheng Xie, Yupeng Zhang, Dawn Song. Transparent Polynomial Delegation and Its Applications to Zero Knowledge Proof. IEEE Symposium on Security and Privacy 2020 / IACR ePrint 2019/1482. URL: https://eprint.iacr.org/2019/1482

[VirgoRevisit25] Changchang Ding, Yan Huang. Revisiting Virgo: a study of vulnerabilities, limitations, and optimizations. Cybersecurity, 2025. DOI: 10.1186/s42400-025-00450-w. URL: https://link.springer.com/article/10.1186/s42400-025-00450-w

[Aurora19] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, Nicholas P. Ward. Aurora: Transparent Succinct Arguments for R1CS. EUROCRYPT 2019 / IACR ePrint 2018/828. URL: https://eprint.iacr.org/2018/828

[StarknetSHARP] Starknet Documentation. SHARP. Accessed 2026-04-23. URL: https://docs.starknet.io/learn/protocol/sharp

[StarkWareSTARK] StarkWare. STARK Technology. Accessed 2026-04-23. URL: https://starkware.co/stark/

[EthereumZKRollups] ethereum.org. Zero-knowledge rollups. Accessed 2026-04-23. URL: https://ethereum.org/developers/docs/scaling/zk-rollups/